Page No.
Date : / /

$\sigma(No) = \frac{e^{z_1}}{e^{z_1}+e^{z_2}+e^{z_3}}$ $\sigma(Yes) = \frac{e^{z_2}}{e^{z_1}+e^{z_2}+e^{z_3}}$ $\sigma\binom{opt}{out} = \frac{e^{z_3}}{e^{z_1}+e^{z_2}+e^{z_3}}$

~~= 0.4~~

for now: $= 0.4$ $= 0.35$ $= 0.25$.

Here His placement's won't be happen coz. $0.4 > 0.35 > 0.25$.

$\sigma(No) > \sigma(Yes) > \sigma(opt\ out)$.

For bigger dataset of higher classes. It will be slower.

Soft Max Loss Function

$$L = \frac{-1}{m} \sum Y_i \log(\hat{Y}_i) + (1-Y_i)\log(1-\hat{Y}_i)$$

For softmax: $L = \frac{-1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} \cancel{-y_k \log(\hat{y}_k)}\ y_k^{(i)} \cdot \log \hat{y}_k^{(i)}$

Page No.
Date: / /

One Hot Encoding.

	x_1	x_2	y	$y_{k=1}$	$y_{k=2}$	$y_{k=3}$
$i=1$	x_{11}	x_{12}	1	1	0	0
$i=2$	x_{21}	x_{22}	2	0	1	0
$i=3$	x_{31}	x_{32}	3	0	0	1

$$l = \frac{-1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} y_k^{(i)} \log(\hat{y}_k^{(i)})$$

$$= \frac{-1}{m} \sum_{i=1}^{m} \left[y_1^{(i)} \log(\hat{y}_1^{(i)}) + y_2^{(i)} \log(\hat{y}_2^{(i)}) + y_3^{(i)} \log(\hat{y}_3^{(i)}) \right]$$

$$= \frac{-1}{m} \Big[\Big[y_1^{(1)} \log(\hat{y}_1^{(1)}) + y_2^{(1)} \log(y_2^{(1)}) + y_3^{(1)} \log(\hat{y}_3^{(1)}) \Big]$$
$$+ y_1^{(2)} \log(\hat{y}_1^{(2)}) + y_2^{(2)} \log(y_2^{(2)}) + y_3^{(2)} \log(\hat{y}_3^{(2)})$$
$$+ y_1^{(3)} \log(\hat{y}_1^{(3)}) + y_2^{(3)} \log(y_2^{(3)}) + y_3^{(3)} \log(\hat{y}_3^{(3)}) \Big]$$

Other will be zero.

~~When $y_1^{(1)} \rightarrow k=1$ & $i=1$.~~ $\rightarrow 1$.

$y^{(1)} \rightarrow i=1$
$y_1 \rightarrow k=1$.

~~$y_2^{(1)}$ other will be zero~~

$$l = \frac{-1}{m} \left[y_1^{(1)} \log(\hat{y}_1^{(1)}) + y_2^{(2)} \log(\hat{y}_2^{(2)}) + y_3^{(3)} \log(\hat{y}_3^{(3)}) \right]$$

Page No.
Date: / /

Now, we have to find
$\hat{y}_1^{(1)}, \hat{y}_2^{(2)}, \hat{y}_3^{(3)}$.

$$\hat{y}_1^{(1)} = \sigma(z)$$
$$z = w_1^{(1)} x_{11} + w_2^{(1)} x_{12} + w_0^{(1)}$$

$$\hat{y}_2^{(2)} = \sigma(z)$$
$$z = w_1^{(2)} x_{21} + w_2^{(2)} x_{22} + w_0^{(2)}$$

$$\hat{y}_3^{(3)} = \sigma(w_1^{(3)} x_{31} + w_2^{(3)} x_{32} + w_0^{(3)})$$

$$\begin{bmatrix} w_1^{(1)} & w_2^{(1)} & w_0^{(1)} \\ w_1^{(2)} & w_2^{(2)} & w_0^{(2)} \\ w_1^{(3)} & w_2^{(3)} & w_0^{(3)} \end{bmatrix}.$$

If Gradient Descent we have to find 9 derivatives.
$\frac{\partial L}{\partial w_1^{(1)}}, \frac{\partial L}{\partial w_2^{(2)}} \ldots$

- Initialize
- loop → 1000 epoch.
- Update

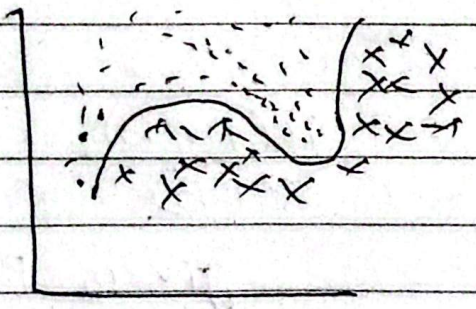
$$w_1^{(1)} = w_{1\,old}^{(1)} - \eta \frac{\partial L}{\partial w_1^{(1)}}$$
$$\vdots$$
9.

Page No.
Date : / /

```
clf = LogisticRegression(multi_class='multinomial')
```

```
query = np.array([[3.4, 2.7]])
clf.predict_prob(query)
clf.predict(query)
```

Non Linear Logistic Regression



In real world, Decision Tree and Random Forest performs better in non linear than logistic regression, but it is better to get informed about it.

[~ like polynomial linear regression]

degree = 2.

$x_1 \quad x_2$

$x_1^0 \quad x_1^1 \quad x_1^2 \quad x_2^0 \quad x_2^1 \quad x_2^2$

$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow$

$w_1 \quad w_2 \quad w_3 \quad w_4 \quad \cdots \quad w_0$

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=3, include_bias=False)
X_trf = poly.fit_transform(X)
clf1 = LogisticRegression()
np.mean(cross_val_score(clf1, X_trf, y, scoring='accuracy',
        cv=10))
```